

Neural Networks for NLP Applications

test document.

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!" } \in \text{ doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	$\ln(\text{word count of doc})$	$\ln(66) = 4.19$

Let's assume for the moment that we've already learned a real-valued weight for

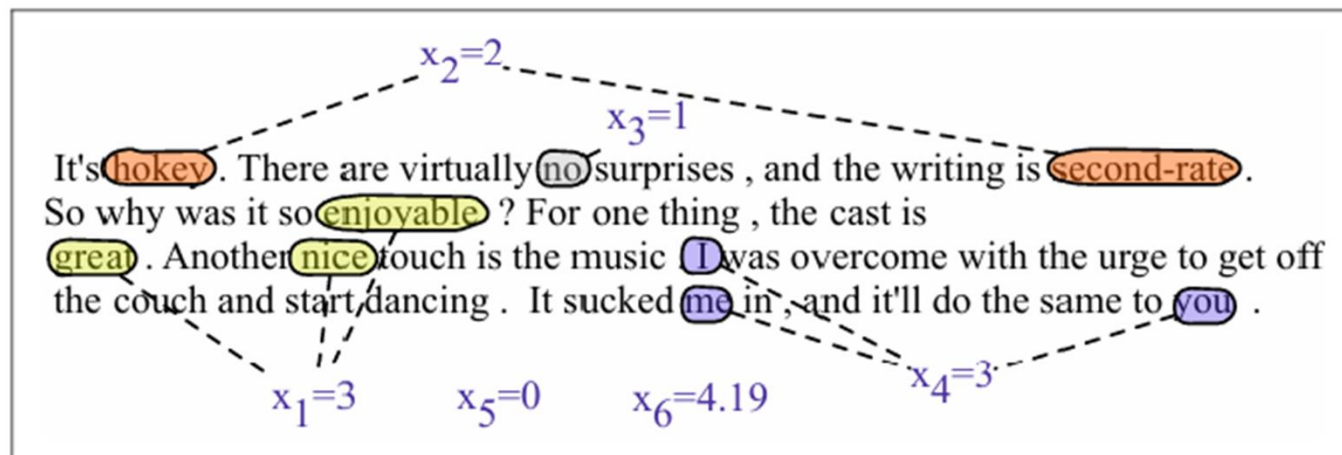


Figure 5.2 A sample mini test document showing the extracted features in the vector x .

Feedforward networks for NLP: Classification

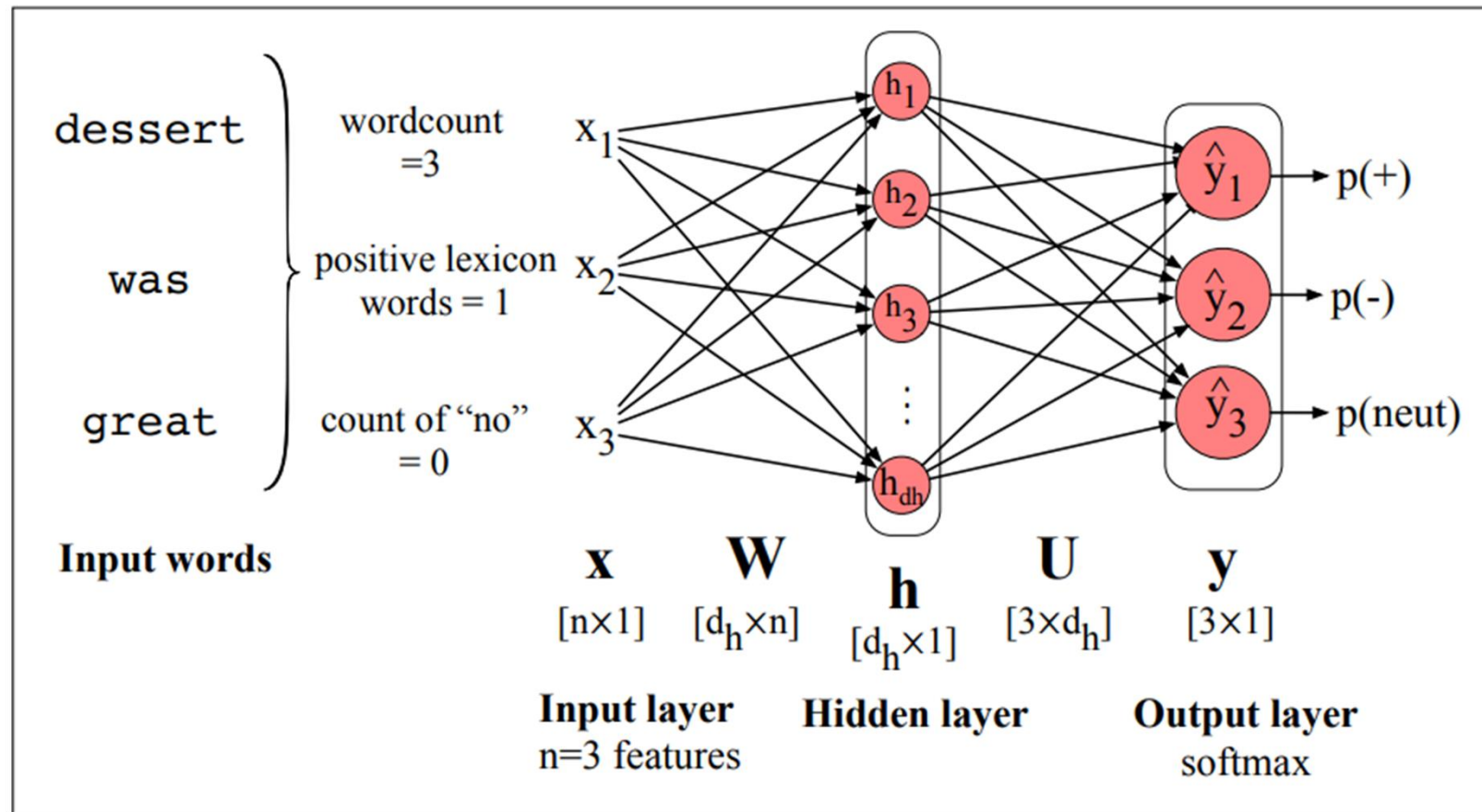


Figure 7.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

Examples Contd..

- Sentiment analysis = figuring out whether a sentence expresses **positive, negative, or neutral** feelings.
- Input Layer (x)

Instead of feeding raw words, we convert the sentence into **hand-built features** (simple numeric values):

- **Wordcount** = 3 (sentence has 3 words: "dessert", "was", "great") $\rightarrow x_1 = 3$
- **Positive lexicon words** = 1 (only "great" is in a dictionary of positive words) $\rightarrow x_2 = 1$
- **Count of "no"** = 0 (there is no "no" in the sentence) $\rightarrow x_3 = 0$

So input vector is:

$$x = \begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix}$$

Input Features

- Word count = 3
- Positive lexicon words = 1
- Count of “no” = 0

$$x = \begin{bmatrix} 3 \\ 1 \\ 0 \end{bmatrix}, \quad x \in \mathbb{R}^{3 \times 1}$$

Hidden Layer

- Weight matrix: $W \in \mathbb{R}^{d_h \times 3}$
- Hidden activations:

$$h = f(Wx), \quad h \in \mathbb{R}^{d_h \times 1}$$

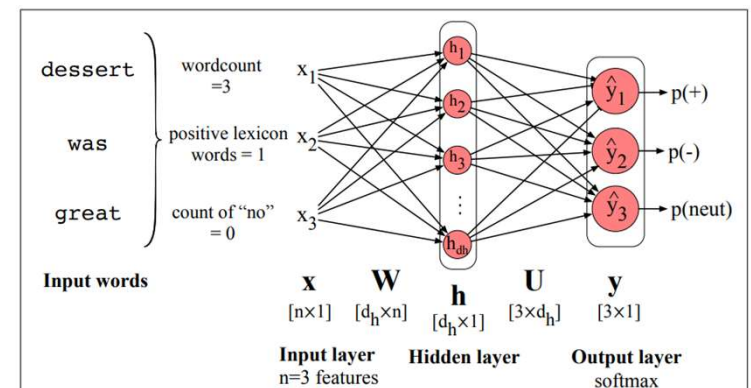


Figure 7.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

2. Weight Matrix (W)

This connects the **input layer** to the **hidden layer**.

- Suppose hidden layer has d_h neurons (say 4, 8, 10, etc., depending on design).
- Then:

$$W \in \mathbb{R}^{d_h \times n} = \mathbb{R}^{d_h \times 3}$$

👉 If $d_h = 4$, then

W is 4×3

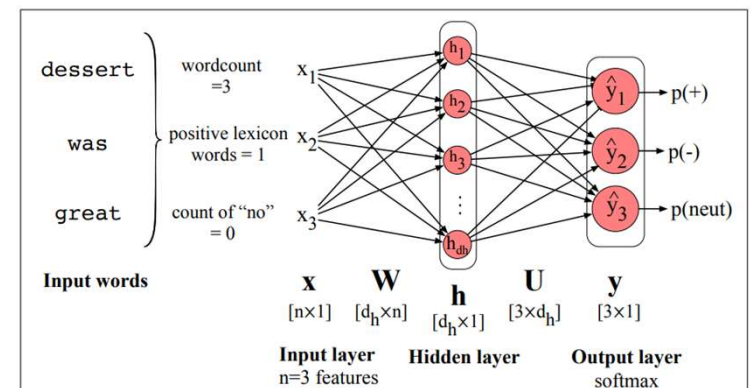


Figure 7.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

3. Hidden Layer (h)

After multiplying:

$$h = f(Wx)$$

(where f = activation function like ReLU, sigmoid, etc.)

- Size of hidden vector:

$$h \in \mathbb{R}^{d_h \times 1}$$

👉 If $d_h = 4$,

$$h = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{bmatrix}, \quad (\text{size } 4 \times 1)$$

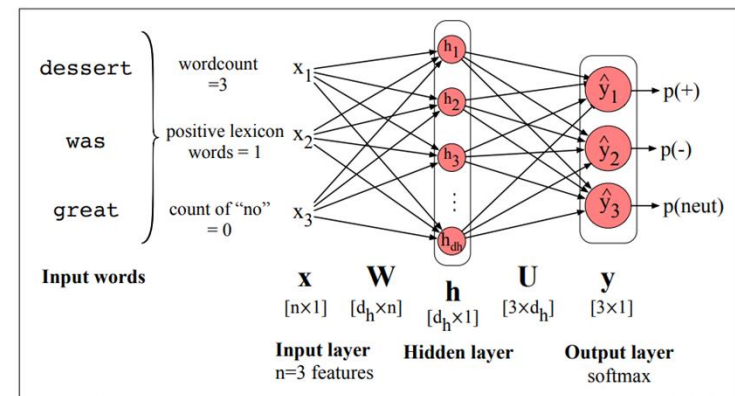


Figure 7.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

4. Weight Matrix (U)

This connects **hidden layer** → **output layer**.

- Output layer has 3 neurons (positive, negative, neutral).
- Hidden layer has d_h neurons.
- So:

$$U \in \mathbb{R}^{3 \times d_h}$$

👉 If $d_h = 4$:

U is 3×4

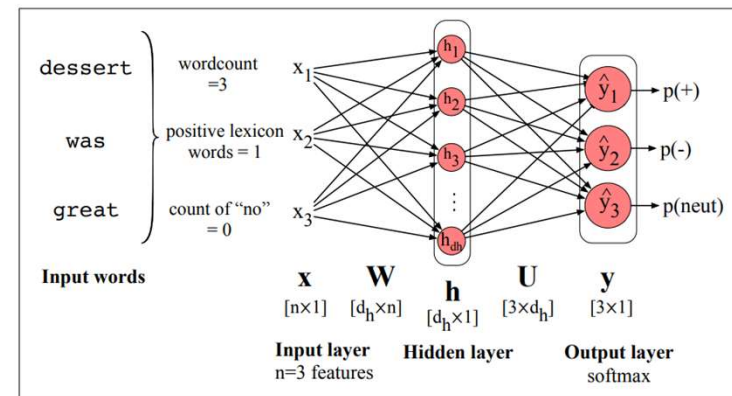


Figure 7.10 Feedforward network sentiment analysis using traditional hand-built features of the input text.

various ways to represent an input for classification

- There are various ways to represent an input for classification.
- One simple baseline is to apply some sort of pooling function to the embeddings of all the words in the input.
- For example, for a text with n input words/tokens $w_1, \dots, w_n$, we can turn the n embeddings $e(w_1), \dots, e(w_n)$ (each of dimensionality d) into a single embedding also of dimensionality d by just summing the embeddings, or by taking their mean (summing and then dividing by n):

$$\mathbf{x}_{mean} = \frac{1}{n} \sum_{i=1}^n \mathbf{e}(w_i) \quad (7.20)$$

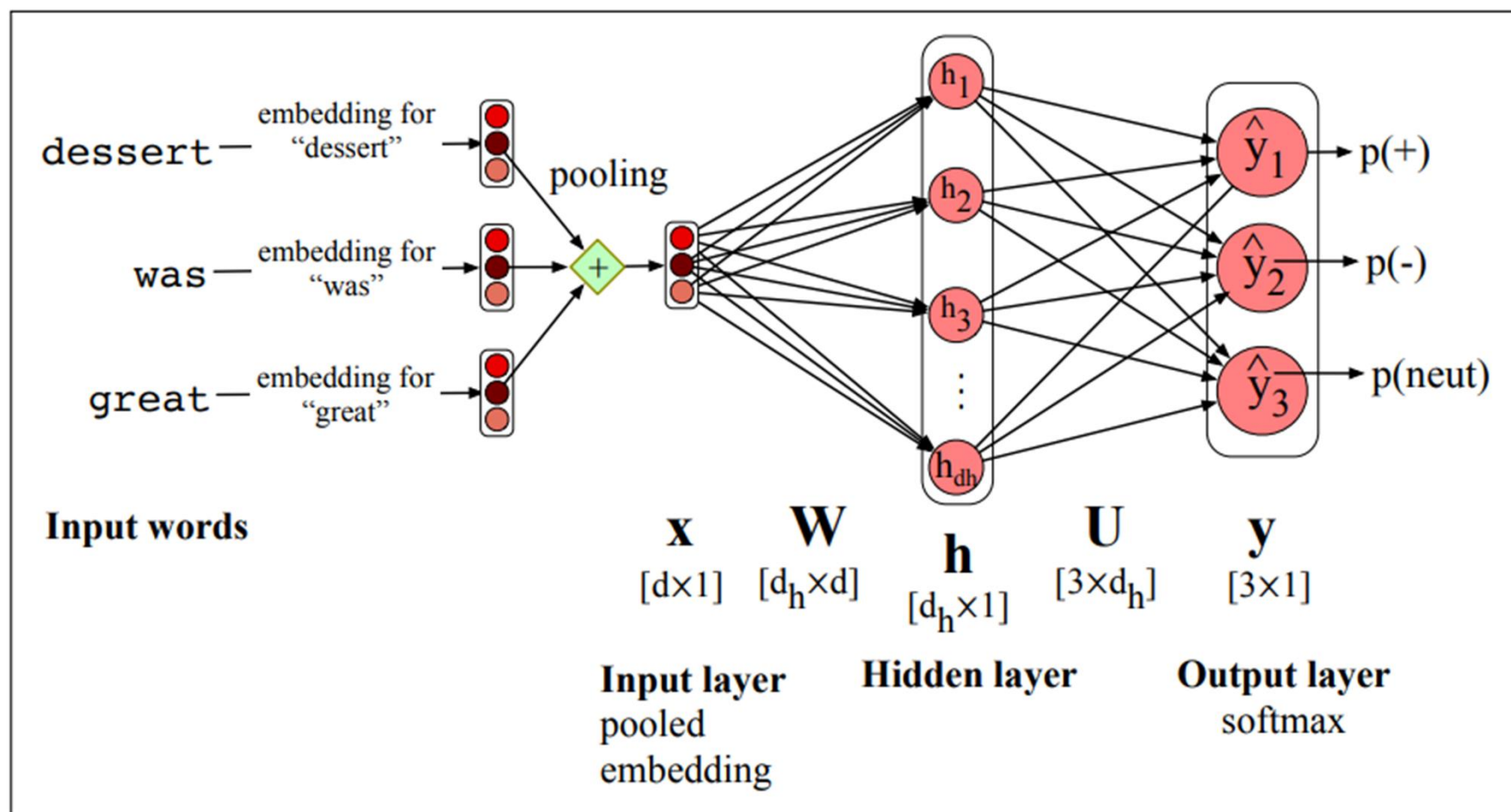


Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the input words.

Q. Consider the feedforward sentiment model with pooled word embeddings shown in Figure 7.11. For the sentence “dessert was great” the following values are given:

Word embeddings (dimension $d = 4$):

- dessert = $[0.10, 0.30, 0.50, 0.20]^T$
- was = $[0.00, 0.10, 0.40, 0.10]^T$
- great = $[0.70, 0.20, 0.60, 0.90]^T$

Pooling: use **average pooling** across the 3 word embeddings to form input vector x .

Hidden layer size: $d_h = 3$.

Weight matrices (given):

$$W = \begin{bmatrix} 0.2 & 0.1 & 0.3 & 0.5 \\ 0.4 & 0.2 & 0.1 & 0.0 \\ 0.1 & 0.3 & 0.2 & 0.4 \end{bmatrix} \in \mathbb{R}^{3 \times 4}, \quad U = \begin{bmatrix} 0.5 & 0.2 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ 0.3 & 0.1 & 0.5 \end{bmatrix} \in \mathbb{R}^{3 \times 3}.$$

Activations:

- Hidden layer uses **sigmoid**: $\sigma(t) = \frac{1}{1 + e^{-t}}$.

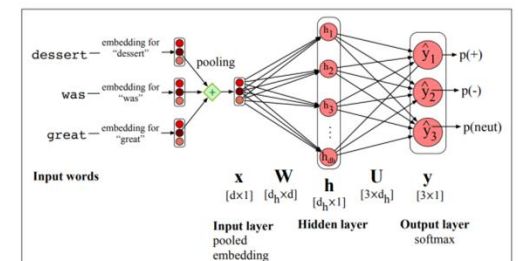


Figure 7.11 Feedforward network sentiment analysis using a pooled embedding of the in-

Solutions....

1. Average pooling $\rightarrow$ input vector x :

$$x = \frac{1}{3} ([0.10, 0.30, 0.50, 0.20]^T + [0.00, 0.10, 0.40, 0.10]^T + [0.70, 0.20, 0.60, 0.90]^T)$$

$$x = \frac{1}{3} [0.80, 0.60, 1.50, 1.20]^T = [0.266666..., 0.2, 0.5, 0.4]^T$$

(round to 2–3 dp:)

$$x \approx \begin{bmatrix} 0.27 \\ 0.20 \\ 0.50 \\ 0.40 \end{bmatrix}, \quad x \in \mathbb{R}^{4 \times 1}.$$

2. Hidden pre-activation $z_h = Wx$: (compute matrix product)

$$z_h = \begin{bmatrix} 0.2 & 0.1 & 0.3 & 0.5 \\ 0.4 & 0.2 & 0.1 & 0.0 \\ 0.1 & 0.3 & 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 0.27 \\ 0.20 \\ 0.50 \\ 0.40 \end{bmatrix} = \begin{bmatrix} 0.41 \\ 0.24 \\ 0.37 \end{bmatrix} \quad (z_h \in \mathbb{R}^{3 \times 1}).$$

Solutions....

3. Hidden activations $h = \sigma(z_h)$ (sigmoid elementwise):

$$\sigma(0.41) \approx 0.6005 \rightarrow 0.601$$

$$\sigma(0.24) \approx 0.5595 \rightarrow 0.559$$

$$\sigma(0.37) \approx 0.5915 \rightarrow 0.592$$

So

$$h \approx \begin{bmatrix} 0.601 \\ 0.559 \\ 0.592 \end{bmatrix}, \quad h \in \mathbb{R}^{3 \times 1}.$$

Solutions....

4. Output logits $z = Uh$:

$$z = \begin{bmatrix} 0.5 & 0.2 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ 0.3 & 0.1 & 0.5 \end{bmatrix} \begin{bmatrix} 0.601 \\ 0.559 \\ 0.592 \end{bmatrix} \approx \begin{bmatrix} 0.4715 \\ 0.4021 \\ 0.5322 \end{bmatrix}.$$

Softmax probabilities (compute e^{z_i} and normalize):

$$e^z \approx [1.6028, 1.4950, 1.7032], \quad \sum \approx 4.8010$$

$$\hat{y} \approx \begin{bmatrix} 1.6028/4.8010 \\ 1.4950/4.8010 \\ 1.7032/4.8010 \end{bmatrix} \approx \begin{bmatrix} 0.334 \\ 0.311 \\ 0.355 \end{bmatrix}.$$

5. Final prediction: highest probability is **neutral** (≈ 0.355).

So the model predicts **Neutral** sentiment.

Why softmax function?

- Logits z_i can be **negative** or **zero**.
- Probabilities must be **non-negative** and **between 0 and 1**.

If you directly normalize z_i , some probabilities might turn negative or zero — which makes no sense for probability distributions.

✓ The exponential e^{z_i} fixes this:

- $e^{z_i} > 0$ for all z_i .
- Ensures smooth positive outputs.
- Naturally emphasizes larger logits (higher confidence).

Thank You